

集成开发阶段题面

之前同学们已经实现了sparse attn和page attn，做到功能正确。但集成到大模型里还需要额外做一些补充，使得算子能适配模型

本阶段所需文件已共享至5-Interate_to_vllm中vllm文件：

```
cp -r /5-Interate_to_vllm/integration/vllm-workspace /
```

创建vllm-ascend环境后先替换环境中默认的

本题由助教提供的测试脚本/文件均在共享文件/vllm-workspace/mycode中

你需要实现的算子均在目录文件/vllm-workspace/vllm-ascend/vllm_ascend/ops/triton_ops.py（只需完成该文件即可实现集成阶段的算子调用）

如若感兴趣同学们可自行研究vllm-ascend文件结构

进阶算子 Reshape and Cache

在大语言模型（LLM）的推理过程中，Paged Attention 机制通过将 KV Cache 划分为物理块（block）来解决显存碎片化问题。当模型生成新的 token 时，需要将新产生的 Key 和 Value 向量准确地写入到预先分配好的物理缓存中。

你的任务是实现一个 **Reshape and Cache** 的前向算子。该算子的核心逻辑是：根据提供的物理槽位映射表（slot mapping），将连续排列的输入 Key/Value 张量“散布”（scatter）到非连续的物理块缓存中。

数据布局定义

- **query**: [num_seqs, num_heads, head_size]
- **key / value**: [num_tokens, num_heads, head_dim]
 - 表示当前推理步产生的待写入缓存的 Key 和 Value 张量。
- **key_cache / value_cache**: [num_blocks, block_size, num_heads, head_dim]
 - num_blocks: 总物理块数。
 - block_size: 每个物理块能容纳的 token 数量。
- **num_kv_heads**: k和v的头的数量
- **slot_mapping**: [num_tokens]

- 含义：slot_mapping[i] 表示第 i 个 token 对应的全局物理槽位索引（Slot Index）。
- 计算逻辑：
 1. block_idx = slot_idx // block_size (确定该 token 属于哪个物理块)
 2. block_offset = slot_idx % block_size (确定该 token 在物理块内的偏移位置)
- 特殊处理：若 slot_mapping[i] < 0，则跳过该 token 的写入。

实现目标

你需要实现函数 `reshape_and_cache(key, value, key_cache, value_cache, slot_mapping)`。
该函数应完成以下逻辑：

1. 遍历每一个待处理的 token 和每一个 head。
2. 通过 `slot_mapping` 获取目标物理位置。
3. 将 `key[token_idx, head_idx, :]` 写入到 `key_cache[block_idx, block_offset, head_idx, :]` 中。
4. 将 `value[token_idx, head_idx, :]` 写入到 `value_cache[block_idx, block_offset, head_idx, :]` 中。

要求：

- **功能正确**：需与 `torch_npu` 原生实现结果一致。
- **性能建议**：在实现中考虑并行化（例如每个 token 或每个 head 作为一个线程/程序块）。

注意：你可以适当的合理修改给出的框架中的内容

进阶算子：GQA Paged Attention

在 LLM 推理的 **Decoding 阶段**，为了高效处理变长序列并减少显存碎片，Paged Attention 允许 KV Cache 在物理内存中不连续存储。本题要求实现一个支持 **Grouped Query Attention (GQA)** 的高性能 Paged Attention 算子。

与综合算子相比，本题引入了**GQA**支持、**在线 Softmax (Online Softmax)** 算法。你需要通过查表获取物理块地址，并在有限的显存带宽下完成注意力计算。在本题中请尽可能优化你的算子，其实取得更高的性能表现。

数据布局定义

- **query**: [num_seqs, num_heads, head_size]
- **key_cache / value_cache**: [num_blocks, block_size, num_kv_heads, head_size]
- **block_tables**: [num_seqs, max_num_blocks]
 - 存储每个序列对应的物理块 ID。
- **context_lens**: [num_seqs]
 - 每个序列当前的实际上下文长度。
 - scale: 缩放因子。

实现目标

1. **GQA 支持**: `num_heads` 可能是 `num_kv_heads` 的整数倍。多个 Query Head 共享同一组 KV Head。
2. **在线 Softmax (FlashAttention 风格)**:
3. **访存逻辑**:
根据 `block_tables` 索引物理块, 并根据 `context_lens` 处理最后一个块的边界掩码 (Masking), 防止读取到无效 Token。
4. 实现函数 `paged_attention(query, key_cache, value_cache, block_tables, context_lens, scale) -> output`
 - **输出**: [num_seqs, num_heads, head_size]
 - **精度**: 与 `torch_npu` 原生算子误差在 $1e-2$ 范围内 (由于 Softmax 累加顺序不同, 允许微小浮点误差)。

要求:

- 功能正确: 需与 `torch_npu` 原生实现结果一致。

注意: 你可以适当的合理修改给出的框架中的内容

Sparse attention算子题面

题目简介

本实验要求实现一个生产级的稀疏注意力算法, 采用分页 KV Cache (Paged KV Cache) 的内存管理策略。整体计算流程与综合算子开发阶段的 Sparse Attention 相

同，但需适配真实推理系统的以下需求：

1. **动态序列长度支持**：每个序列可具有不同长度，通过 `context_lens` 参数指定
2. **分页内存管理**：KV Cache 以物理块形式存储，通过 `block_table` 进行逻辑块到物理块的映射

数据布局定义

参数	形状	说明
<code>query</code>	<code>[batch_size, num_q_heads, head_dim]</code>	当前解码步的查询向量
<code>key_cache</code>	<code>[num_blocks, block_size, num_kv_heads, head_dim]</code>	物理块优先的 Key Cache
<code>value_cache</code>	<code>[num_blocks, block_size, num_kv_heads, head_dim]</code>	物理块优先的 Value Cache
<code>block_table</code>	<code>[batch_size, max_blocks_per_seq]</code>	逻辑块 → 物理块映射表
<code>context_lens</code>	<code>[batch_size]</code>	每个序列的实际长度
<code>out</code>	<code>[batch_size, num_q_heads, head_dim]</code>	输出注意力结果

参数说明：

参数	说明
<code>num_kv_heads</code>	KV 头数量
<code>num_heads</code>	Q 头数量（应满足 $\text{num_heads} = \text{num_kv_heads} \times \text{q_per_k}$ ）
<code>scale_value</code>	注意力缩放因子
<code>block_size</code>	每个物理块包含的 token 数
<code>topk_blocks</code>	每个 KV 头选择的逻辑块数量
<code>max_blocks_per_seq</code>	每个序列最多可分配的逻辑块数量 (由 <code>block_table.shape[1]</code> 决定)

算法要求

实现思路与综合算子开发阶段的 Sparse Attention 保持一致，核心流程如下：

1. **评分阶段**：计算每个 Query 对每个逻辑块的评分（基于 Quest 近似方法）
2. **筛选阶段**：选择评分最高的 `topk_blocks` 个逻辑块
3. **注意力阶段**：仅对选出的逻辑块执行完整的 GQA 注意力计算

注意：需使用 `context_lens` 判断逻辑块是否有效（即块内 token 是否超出序列实际长度）。

实现目标

实现以下接口函数，功能正确性要求与参考实现（先评分 + Top-K + 精确注意力）的结果误差在 `1e-2` 以内。

python

```
def sparse_paged_attention(
    query: torch.Tensor,                # [B, Q_H, D]
    key_cache: torch.Tensor,            # [num_blocks, block
    _size, KV_H, D]
    value_cache: torch.Tensor,          # [num_blocks, block
    _size, KV_H, D]
    num_kv_heads: int,
    num_heads: int,
    scale_value: float,
    block_table: torch.Tensor,          # [B, max_blocks_per
    _seq]
    context_lens: torch.Tensor,         # [B]
    out: torch.Tensor,                  # [B, Q_H, D]
    block_size: int,
    topk_blocks: int,
    **kwargs,
) -> None:
    """
    Sparse PagedAttention 算子接口

    实现要求：
    1. 支持 GQA (Grouped Query Attention)
    2. 使用 context_lens 处理变长序列
    3. 通过 block_table 实现逻辑块到物理块的映射
    4. 仅对 topk_blocks 个逻辑块执行完整注意力计算
    """
    pass
```

同学们可以自行优化如进行算子融合等方案优化性能，只需保持算子对外暴露的接口不变即可

在本阶段任务中，鼓励同学们充分发挥创造力，尽可能优化算子实现以获得更优的性能表现。

Triton 算子的性能将作为最终评奖的重要标准。评奖时会综合考虑以下方面：

- **计算效率**：合理设计 Grid 与 Block 大小，充分利用硬件并行能力
- **访存优化**：减少冗余内存访问，提高数据局部性
- **算子融合**：在保证正确性的前提下，适当融合计算步骤
- **硬件适配**：针对 NPU 架构特点（如 65536 Grid 限制、数据类型对齐等）进行针对性优化

期待同学们能够设计出兼具正确性与高性能的 Triton 算子实现。

triton_ops.py代码结构

本文件中的每个 Triton 算子采用**三层结构**进行组织，以 `PagedAttention` 为例说明如下：

1. 内核函数（`_xxx_kernel`）

- 使用 `@triton.jit` 装饰器定义
- 包含具体的 NPU 线程级实现逻辑，完成算子的核心计算

2. 算子封装类（`TritonXxx`）

- 命名如 `TritonPagedAttention`
- 在 `__init__` 方法中绑定对应的内核函数
- 提供 `forward` 方法，负责：
 - 参数校验与预处理
 - 计算网格大小（grid）和分块参数
 - 调用内核函数完成实际计算
 - 处理返回值

3. 对外接口函数（`triton_xxx`）

- 命名如 `triton_paged_attention`
- 作为模块暴露的统一入口
- 内部通过单例模式（如 `TritonPagedAttention()`）获取算子类实例
- 调用实例的 `forward` 方法，将参数透传

测试文件说明

本题要求所实现算子通过所有正确性测试，并尽可能地提升算子性能。

测试前先执行

```
export PATH=/usr/local/python3.11.14/bin:$PATH
```

```
export VLLM_USE_MODELSCOPE=true
```

- `compare_paged_attention.py`

PagedAttention 算子的正确性测试文件。运行该文件，若输出结果为 `passed`，则说明你实现的 PagedAttention 算子正确性通过。

```
=====
手写 Triton 算子 vs 官方 Ascend 算子 评测
=====
query: shape=torch.Size([4, 32, 128]), dtype=torch.float16
key_cache: shape=torch.Size([2048, 128, 8, 128]), dtype=torch.float16
value_cache: shape=torch.Size([2048, 128, 8, 128]), dtype=torch.float16
num_kv_heads: 8
num_heads: 32
scale_value: 0.08838834764831843
block_table: shape=torch.Size([4, 64]), dtype=torch.int32
context_lens: shape=torch.Size([4]), dtype=torch.int32
block_size: 128
max_seq_len: 8192
head_dim: 128
=====
1. 正确性测试 (NPU Native Triton vs NPU 官方)
=====
最大误差: 0.000122
平均误差: 0.000005
Status: ✓ PASSED
```

- `compare_reshapeandcache.py`

ReshapeAndCache 算子的正确性测试文件。运行该文件，若输出结果为 `passed`，则说明你实现的 ReshapeAndCache 算子正确性通过。

```
-----Initializing custom ops.-----
Max difference Key: 0.0
Max difference Value: 0.0
Status: ✓ PASSED
```

- `compare_sparse_paged_attention.py`

Sparse Attention 算子的正确性测试文件。运行该文件，若输出结果为 `passed`，则说明你实现的 Sparse Attention 算子正确性通过。

*这里有一个PyTorch版本的Sparse实现，根据同学们提交的版本的误差，再将其列入正确性或者精度评判中

```
=====
Sparse Paged Attention Triton 测试
=====
query: shape=torch.Size([4, 32, 128]), dtype=torch.float16
key_cache: shape=torch.Size([16384, 32, 8, 128]), dtype=torch.float16
value_cache: shape=torch.Size([16384, 32, 8, 128]), dtype=torch.float16
num_kv_heads: 8
num_heads: 32
scale_value: 0.08838834764831843
block_table: shape=torch.Size([4, 128]), dtype=torch.int32
context_lens: shape=torch.Size([4]), dtype=torch.int32
block_size: 32
max_seq_len: 4096
max_blocks_per_seq: 128
head_dim: 128
valid_blocks: [89, 118, 113, 104]

=====
1. 正确性测试 (Sparse topk=full vs 官方 Ascend Dense)
=====
topk_blocks: 128
最大误差: 0.000061
平均误差: 0.000005
Status: ✓ PASSED

=====
2. Sparse 近似误差测试
=====
context_lens: [181042, 240100, 231171, 212119]
valid_blocks: [5658, 7504, 7225, 6629]
平均有效块数: 6754.00
稀疏 topk_blocks: 8
最大误差: 0.530273
平均误差: 0.083496
```

- `Sparse_compare1.sh`

测试性能指标

样例输出:

===== 性能对比报告 (Triton vs NPU vs Sparse)

=====

Metric | TRITON | NPU | SPARSE | SPARSE/NPU | SPARSE/Triton

Requests/s | 1.35 | 2.43 | 0.76 | 0.3128 | 0.5630

Total Tokens/s | 612.71 | 1100.29 | 346.48 | 0.3149 | 0.5655

Output Tokens/s | 298.25 | 535.59 | 168.66 | 0.3149 | 0.5655

Token Statistics:

[Triton] Total num prompt tokens: 23260

[NPU] Total num prompt tokens: 23260
[Sparse] Total num prompt tokens: 23260

- `Sparse_compare2.sh`

测试性能指标

样例输出:

```
===== Serving Benchmark Result =====
Successful requests: 1000
Failed requests: 0
Benchmark duration (s): 380.24
Total input tokens: 217393
Total generated tokens: 201847
Request throughput (req/s): 2.63
Output token throughput (tok/s): 530.84
Peak output token throughput (tok/s): 3317.00
Peak concurrent requests: 1000.00
Total token throughput (tok/s): 1102.57
-----Time to First Token-----
Mean TTFT (ms): 71358.02
Median TTFT (ms): 72571.68
P99 TTFT (ms): 179769.82
-----Time per Output Token (excl. 1st token)-----
Mean TPOT (ms): 310.82
Median TPOT (ms): 306.20
P99 TPOT (ms): 603.71
-----Inter-token Latency-----
Mean ITL (ms): 340.29
Median ITL (ms): 77.82
P99 ITL (ms): 814.99
```

- `bench_compare.sh`

用于测试 ReshapeAndCache 算子和 PagedAttention 算子综合性能的脚本。

```
root@nb-a19110696532832256249851-a19110696532832256249851-0:/vllm-workspace/mycode# source bench_compare.sh
正在运行: TRITON_MODE ...
正在运行: NPU_MODE ...

===== 性能对比报告 (Triton vs NPU) =====
Metric | TRITON | NPU | Ratio (T/N)
-----|-----|-----|-----
Requests/s | 11.02 | 12.37 | 0.8909
Total Tokens/s | 4620.99 | 5185.88 | 0.8911
Output Tokens/s | 2224.82 | 2496.79 | 0.8911
=====
Token Statistics:
[Triton] Total num prompt tokens: 217393
[NPU] Total num prompt tokens: 217393
```

- `example.py`

用于测试 LLM 在不同 prompt 下输出的示例文件。（如你仅想测试你的某个算子的正确性可按如下指定环境变量进行测试）

通过指定环境变量可选择调用不同的算子实现：

```
#仅测试你实现的reshapeandcache算子，pagedattn仍使用官方torch_npu
库的实现
VLLM_KVCACHE_MODE=triton PAGED_ATTN_MODE=npu python example.py
```

环境变量	取值	说明
VLLM_KVCACHE_MODE	npu	调用官方 <code>torch_npu</code> 的 ReshapeAndCache 算子
	triton	调用你自己实现的 ReshapeAndCache 算子
PAGED_ATTN_MODE	npu	调用官方 <code>torch_npu</code> 的 PagedAttention 算子
	triton	调用你自己实现的 PagedAttention 算子
	sparse	调用你自己实现的 Sparse Attention 算子

默认行为：不指定变量时，`VLLM_KVCACHE_MODE=npu`，`PAGED_ATTN_MODE=npu`。

pagedattn and reshape算子综合性能测试示例

你也可以通过 vLLM 的吞吐量测试脚本，综合验证算子的性能表现：

```
VLLM_KVCACHE_MODE=triton \
PAGED_ATTN_MODE=triton \
python -m vllm.entrypoints.cli.main bench throughput \
    --model Qwen/Qwen3-0.6B \
    --dataset-name sharegpt \
    --dataset-path ./ShareGPT_V3_unfiltered_cleaned_split.j
son \
    --num-prompts 1000 \
    --enforce-eager \
    --no-enable-chunked-prefill
```

该命令会使用你实现的 Triton 算子（包括 ReshapeAndCache 和 PagedAttention）进行吞吐量测试，可用于评估算子的实际性能表现。

附加题

若完成以上所有必做题后仍学有余力，可自行调研并完成以下附加题：

实现 Triton 算子的 NPU Graph 入图功能

本部分可能会涉及 `triton_ops.py` 以外文件的修改，同学们可自由发挥。成功完成附加题的同学，可单独提交给助教进行审核，将获得额外的加分。

算子评测与计分逻辑说明

本题目的代码评测分为正确性（Correctness）与性能（Performance）两个维度。评测系统会在 Huawei Ascend NPU 环境下，将您提交的算子实现（如 `attention_v1.py` 等）注入到 VLLM 测试框架中进行端到端检验。

一、评测流程概述

环境准备：系统会将您的提交文件自动拷贝至测试沙箱的 `vllm_ascend/attention/` 等核心目录下覆盖占位文件。

正确性检验：运行基础测试用例，比对您的算子输出与标准 PyTorch 实现的精度误差。

性能基准测试：在确保正确性的前提下，运行吞吐量测试，系统将会测试针对不同场景下的数据，并测试不同的并发请求下的吞吐量。

异常拦截：如果您的代码发生 `RuntimeError`（如显存溢出、算子崩溃）或运行时间过长触发 `TimeoutExpired`，该次评测将直接判为 0 分。

二、正确性评测

在正确性测试环节，评测机会严格检查算子计算结果的最大误差（Max difference）。

评判标准：系统会解析日志中的 `Max difference Key` 和 `Max difference Value`。只有当误差在允许的阈值范围内，且系统输出 `Status: ✓ PASSED` 时，该测试用例才算通过。

计分方式：每个成功 PASSED 的测试用例将获得基础正确性得分，共有 3 个测试用例，具体逻辑可以参考：

- `compare_paged_attention.py`
- `compare_reshapeandcache.py`
- `compare_sparse_paged_attention.py`

三个文件，每一个测试点 33.333 分，全部完成为 100 分；完成 `compare_paged_attention.py` 与 `compare_reshapeandcache.py` 或者完成 `compare_sparse_paged_attention.py` 均可以 `Accept` 并进入到性能评测阶段

三、性能评测

性能评测旨在考察您对底层硬件特性、内存访存以及算子融合的优化能力。系统主要参考以下两个核心加速比（Ratio）指标：

RATIO_1000_TRITON_VS_NPU_TOT：在 1000 并发请求下，您实现的 TRITON 算子总吞吐量对比基线 NPU 实现的加速比。

RATIO_100_SPARSE_VS_NPU_TOT：在 100 并发请求下，稀疏注意力（Sparse Attention）模式对比稠密 NPU 模式的总吞吐量比例。

综合性能得分 (Score Performance) 计算公式：

最终的性能分将通过以下加权公式计算得出：

$$\text{Score} = (0.45 \times \text{RATIO_1000_NPU_VS_TRITON_TOT}) + (0.55 \times \text{RATIO_100_SPARSE_VS_NPU_TOT})$$